



# NotifyHub

## Architecture & Workflow

A centralized backend notification platform that delivers notifications reliably and asynchronously, decoupled from producer business logic — built on the transactional outbox pattern, RabbitMQ, and Server-Sent Events.

Java 21 · Spring Boot 3.4

PostgreSQL

RabbitMQ

React 19 · Vite

Outbox pattern

SSE

API-key auth

Gmail SMTP

Rich HTML email

## What the system is

NotifyHub turns a single "please notify this user" request into a reliable, observable delivery — without blocking or coupling the caller's workflow.

Producers submit a notification intent over REST. NotifyHub persists it as the source of truth, hands delivery to an asynchronous pipeline, and reports the final state ( **SENT** or **FAILED** ) both in the database and live over Server-Sent Events. Delivery failures are retried and, once exhausted, dead-lettered — never lost, never silently dropped.

### Design principles

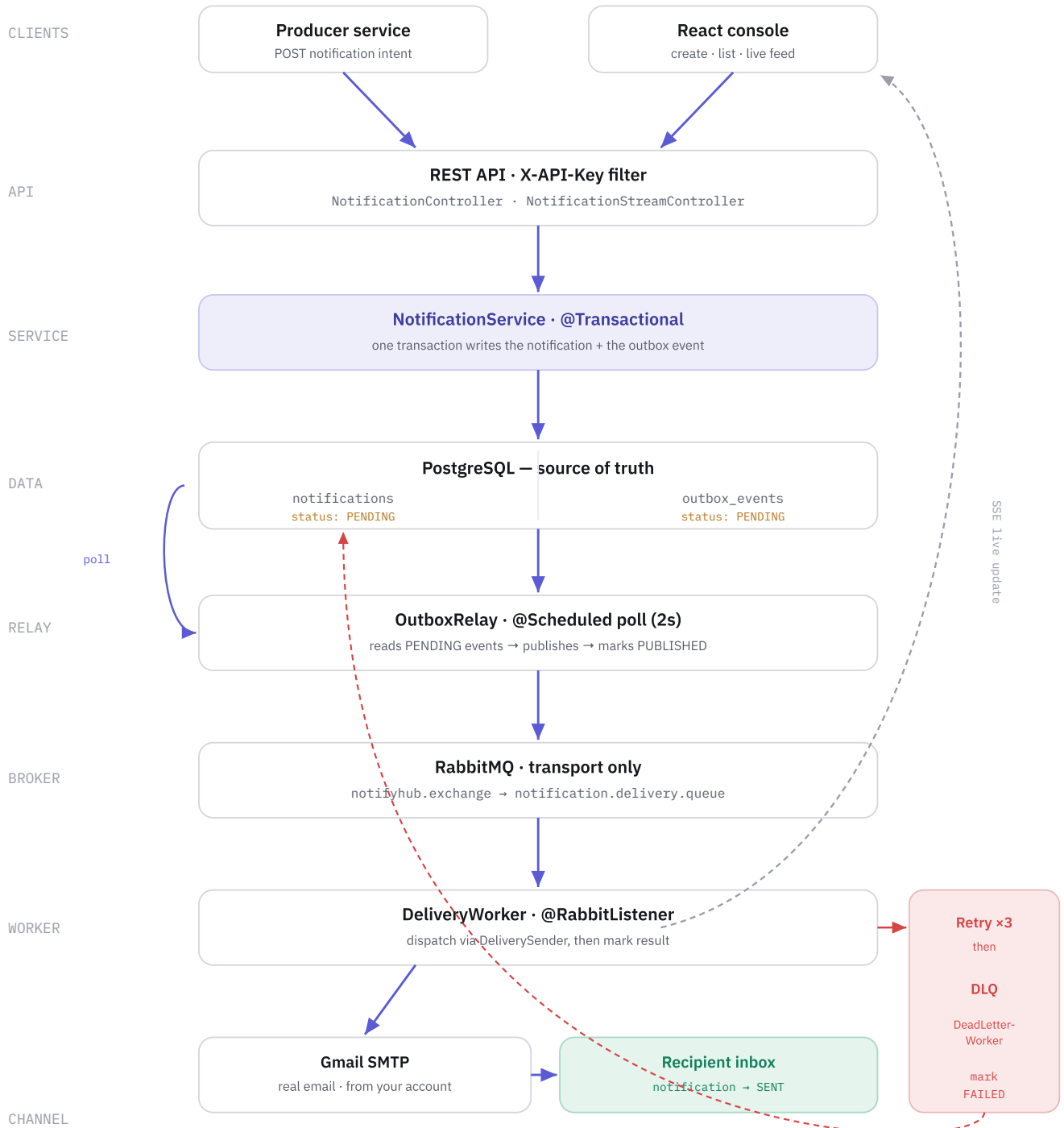
- **Database is the source of truth** — state is persisted explicitly.
- **Asynchronous, non-blocking** — the caller returns immediately.
- **Failure-aware** — bounded retries then explicit dead-lettering.
- **Best-effort real-time** — SSE augments, never replaces, the DB.

### Guarantees

- **No lost events** — outbox write is in the same transaction as the notification.
- **At-least-once delivery** — the relay retries unpublished events.
- **Terminal states are recorded** — DLQ handler marks exhausted sends FAILED.
- **Transport  $\neq$  state** — RabbitMQ carries work; Postgres holds truth.

## How a request travels across layers

Solid arrows = the delivery path. Dashed = live updates & the failure branch. The database sits at the center as the source of truth; RabbitMQ is transport only.



## End to end, step by step

What happens from the moment a producer calls `POST /api/v1/notifications`.

- 1 Authenticate.** The `ApiKeyAuthFilter` checks `X-API-Key` (constant-time). Missing/invalid → `401`. Valid → the request proceeds.
- 2 Validate.** `NotificationController` binds the JSON to a validated DTO (email format, required fields). Violations → `400` with an RFC-7807 body.
- 3 Persist atomically.** `NotificationService.create()` runs in one transaction: it saves the `Notification` as `PENDING` and an `OutboxEvent` carrying the delivery payload. Both commit or neither does.
- 4 Respond immediately.** The API returns `201 Created` with the notification — the caller is never blocked on delivery.
- 5 Relay.** `OutboxRelay` (scheduled, every 2s) finds `PENDING` outbox rows, publishes each to RabbitMQ, and marks them `PUBLISHED`. A failed publish stays `PENDING` and is retried next tick — at-least-once.
- 6 Consume & deliver.** `NotificationDeliveryWorker` (`@RabbitListener`) receives the task and calls the configured `DeliverySender` — a log mock, or real Gmail SMTP that renders a branded HTML email when branding is present (else plain text).
- 7 Record result.** On success the notification becomes `SENT`; the outcome is pushed to any connected SSE clients.
- 8 Failure path.** If the send throws, RabbitMQ retries up to 3× with backoff; still failing, the message is dead-lettered. `DeadLetterWorker` marks it `FAILED` and notifies over SSE.

## Why nothing gets lost

### Transactional outbox

Publishing to a broker and committing to a database can't share one transaction. Instead, the delivery event is written to `outbox_events` in the *same* DB transaction as the notification. A separate relay publishes it afterward. If the app crashes before publishing, the event is still there — it goes out on the next poll.

### Retry & dead-letter

The listener retries a failing delivery 3× with exponential backoff. When attempts are exhausted the message is rejected to a dead-letter exchange and lands on `notification.delivery.dlq`, where `DeadLetterWorker` records the terminal **FAILED** state. Failures are visible, not silent.

| CONCERN               | MECHANISM                                        | OUTCOME                       |
|-----------------------|--------------------------------------------------|-------------------------------|
| Lost publish on crash | Outbox written in the notification's transaction | Event survives; relay resends |
| Broker/publish hiccup | Event stays PENDING, re-pollled                  | At-least-once delivery        |
| Provider outage       | 3× retry with backoff                            | Transient errors self-heal    |
| Permanent failure     | Dead-letter queue + handler                      | Recorded as FAILED            |
| Slow SMTP host        | 10s connect/read/write timeouts                  | Fails fast into retry path    |

# Backend structure

| PACKAGE                 | TYPE               | RESPONSIBILITY                                       |
|-------------------------|--------------------|------------------------------------------------------|
| notification.web        | Controllers / DTOs | REST intake, SSE stream, validation, RFC-7807 errors |
| notification.service    | Service            | Transactional create (notification + outbox), reads  |
| notification.domain     | Entities           | Notification , OutboxEvent + status enums            |
| notification.repository | Spring Data JPA    | Persistence for notifications & outbox               |
| notification.outbox     | Scheduled relay    | OutboxRelay — poll & publish to RabbitMQ             |
| notification.messaging  | Broker             | Rabbit config, delivery worker, dead-letter worker   |
| notification.delivery   | Channel seam       | DeliverySender : logging + Gmail SMTP impls          |
| notification.realtime   | SSE                | SseService , per-recipient emitters                  |
| config                  | Security           | API-key filter + stateless security chain            |

# Frontend structure

| PATH           | CONTENTS                                                                                                                            |
|----------------|-------------------------------------------------------------------------------------------------------------------------------------|
| src/pages      | Console ( / ): pipeline + table + compose + feed. Overview ( /overview ): delivery lane                                             |
| src/lib        | Types, API client (API-key + RFC-7807 aware), formatters                                                                            |
| src/hooks      | useNotifications (TanStack Query), useNotificationStream (SSE)                                                                      |
| src/components | NotificationPipeline (signature view), DeliveryLane, NotificationsTable, ComposeForm (with branding), LiveFeed, NavRail, StatusPill |

Stack: React 19 · Vite · TypeScript · Tailwind v4 · TanStack Query · React Router. Fonts: Space Grotesk, IBM Plex Sans/Mono. The signature view is a per-notification pipeline that animates through stages as the status advances.

## Persistence & endpoints

### notifications

**id** uuid · primary key  
**to\_email** varchar(255)  
**subject** varchar(255)  
**body** text  
**status** PENDING · SENT · FAILED  
**created\_at** · **updated\_at** timestamp  
 + branding (V2): logo\_url, banner\_url, heading, cta\_text, cta\_url

### outbox\_events

**id** uuid · primary key  
**event\_type** varchar(100)  
**payload** jsonb  
**status** PENDING · PUBLISHED · FAILED  
**created\_at** · **published\_at** timestamp  
 index on (status, created\_at)

## HTTP API

| METHOD | PATH                         | PURPOSE                             | AUTH         |
|--------|------------------------------|-------------------------------------|--------------|
| POST   | /api/v1/notifications        | Create a notification intent        | X-API-Key    |
| GET    | /api/v1/notifications        | List notifications                  | X-API-Key    |
| GET    | /api/v1/notifications/{id}   | Fetch one                           | X-API-Key    |
| GET    | /api/v1/notifications/stream | SSE delivery stream (per recipient) | apiKey query |
| GET    | /actuator/health             | Liveness probe                      | public       |

## Access control and the send channel

### API-key authentication

Every endpoint except the health probe requires a valid `X-API-Key` (the SSE stream accepts it as an `apiKey` query param, since `EventSource` can't set headers). Keys come from env, support comma-separated rotation, and are compared in constant time. Security is stateless — no sessions, CSRF disabled.

Scope: platform-level access. Per-recipient authorization (a user seeing only their own stream) would need user identity (JWT) — a documented next step.

### Pluggable delivery

Delivery is a single seam, the `DeliverySender` interface, selected by

`notifyhub.delivery.mode` :

**log** (default) — logs the dispatch; a subject containing "fail" exercises the DLQ path.

**smtp** — real email via Gmail ( `smtp.gmail.com:587` , STARTTLS).

Gmail needs only `EMAIL_USER` +

`EMAIL_APP_PASSWORD` (a 2FA App Password); sent mail's From is that account. With optional branding (logo, banner, heading, CTA) the message is rendered as a responsive, HTML-escaped email template; otherwise it is plain text. Images are referenced by URL.

### Runtime topology

Postgres and RabbitMQ run as Docker containers; the app runs locally (IDE) or as a third container. All configuration is env-driven with local defaults, so the app boots without a hand-written `.env`.



# Delivery log

Phases 2–4 of the platform plus the frontend, security, and real email — each increment committed and verified end-to-end.

| COMMIT            | CHANGE                                                                                      |
|-------------------|---------------------------------------------------------------------------------------------|
| 115081c           | Notification pipeline — intake, persistence, outbox, RabbitMQ worker, DLQ, SSE (Phases 2–4) |
| 8ca0c80           | Restructure into <code>backend/</code> + <code>frontend/</code> monorepo                    |
| 07765c3           | React delivery console (Vite + Tailwind + TanStack Query)                                   |
| 6ade523           | Make datasource/rabbit config runnable from an IDE without env vars                         |
| 7511226           | Real email delivery via SMTP (pluggable <code>DeliverySender</code> )                       |
| 799952f           | Add <code>.env.example</code> template                                                      |
| 0206668           | Require API key on all endpoints                                                            |
| b0d9c7e → b038d86 | SMTP hardening, then pin delivery to Gmail                                                  |
| a2aab67           | JaCoCo + full unit-test suite (100% coverage) and CI on GitHub Actions                      |
| 593e3f6           | Per-notification delivery pipeline with animated stages (frontend)                          |
| 3397cbb           | Split into Console / Overview pages with routing                                            |
| 9b79dc9           | Rich branded HTML emails (logo, banner, heading, CTA)                                       |

## Verified behavior

- Normal notification: PENDING → relay → worker → SENT .
- Fail path: subject with "fail" → retries → DLQ → FAILED .
- Real email delivered through SMTP to a live inbox (verified against a mail catcher).
- Branded HTML email rendered with logo, banner, heading, and CTA button.
- Auth matrix: no key → 401, wrong key → 401, valid key → 200/201, health public.
- 60+ unit tests at 100% coverage; CI green on every push.